Clase 4
Arquitectura de Software

Objetivos de la Clase

- Entender los *principios de arquitectura de software*
- Explorando distintos *estilos arquitectónicos*
- Qué son los *patrones arquitectónicos?*
- Vistazo de *patrones de diseño de software*

Agenda

1. **Introduction and Fundamentals**
2. Architecture Styles
3. Architecture Patterns
4. Software Design Patterns

What is a software architecture?

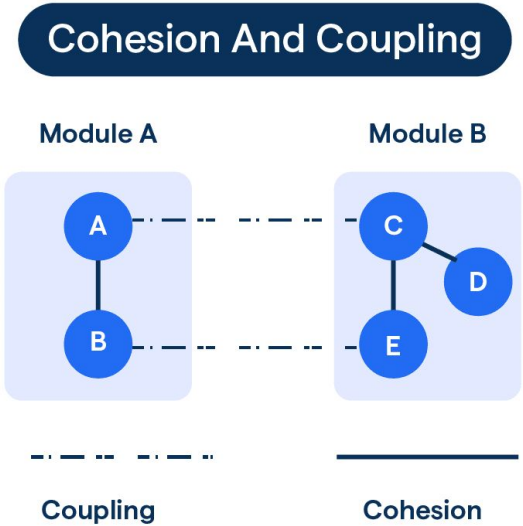
- Is the **collection of design decisions** that shape a system.
- Provides a **shared vocabulary and vision** for developers and stakeholders.
- **Defines both the components and their interactions**

Laws of software architectures

- Everything is a **trade-off**
- No **silver bullet**
- Architecture exists **even if you don’t design it**
- **Why** is more important than how
- Optimize for **change**

Why It Matters?

- Reduce **coupling**, increase **cohesion**
  - Fewer dependencies, focused modules, better structure.
- Enable **autonomy** and **testing**
  - Independent delivery, faster feedback, simpler validation.



Definitions

- Architecture Styles
  - Broad **system organization** approach.
  - Defines structure **philosophy**.
- Architecture Patterns
  - Proven **high-level solutions** for system design
  - Guides **implementation**.
- Software Patterns
  - Reusable **code-level** solutions (e.g., Singleton, Observer).
  - Solves **specific programming problems**.

Definitions

Feature	Architectural Style	Architectural Pattern	Design Pattern
Scope	High-level structure	System-wide solution	Code-level solution
Purpose	Organizes system components	Implements architectural style	Solves specific coding problems
Impact	Broad system-level	Horizontal & vertical system implementation	Localized to specific modules
Examples	Layered, Microkernel, SOA	MVC, Microservices, Message Bus	Singleton, Observer, Strategy





CQRS Architecture

separates read and write operations for a data store. It enables independent scaling of read and write workloads and optimizes them separately.

Orchestration Architecture

a central coordinator (often called an orchestrator) that directs the interaction between services. The orchestrator is responsible for managing the control flow and data flow between services.

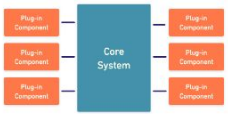


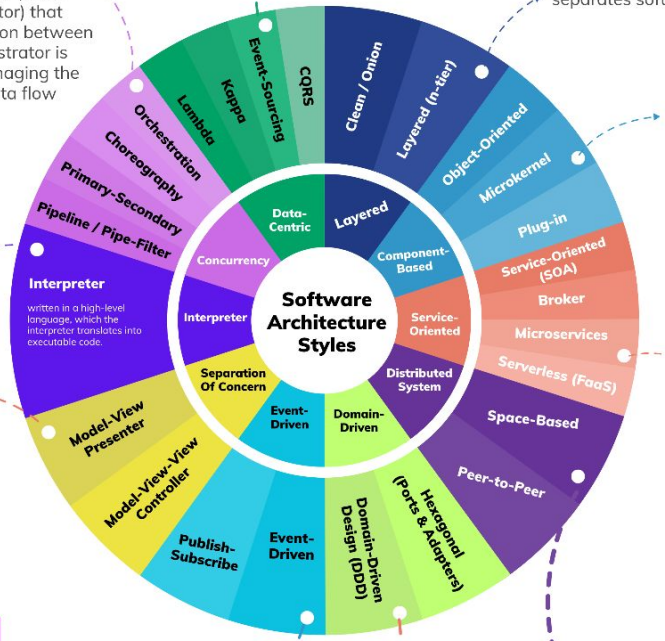
Layered (n-tier) Architecture

separates software into logical layers

Microkernel Architecture

separates a minimal functional core from extended functionality and customer-specific parts.





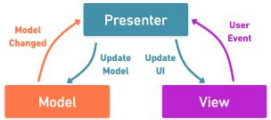
Interpreter

written in a high-level language, which the interpreter translates into executable code.



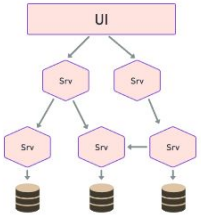
MVP Architecture

derivative of the Model-View-Controller (MVC) pattern, which aims to separate the concerns of data management, user interface, and control flow.



Microservice Architecture

This architecture designs a software application as a suite of independently deployable, small, modular services.



Event-Driven Architecture

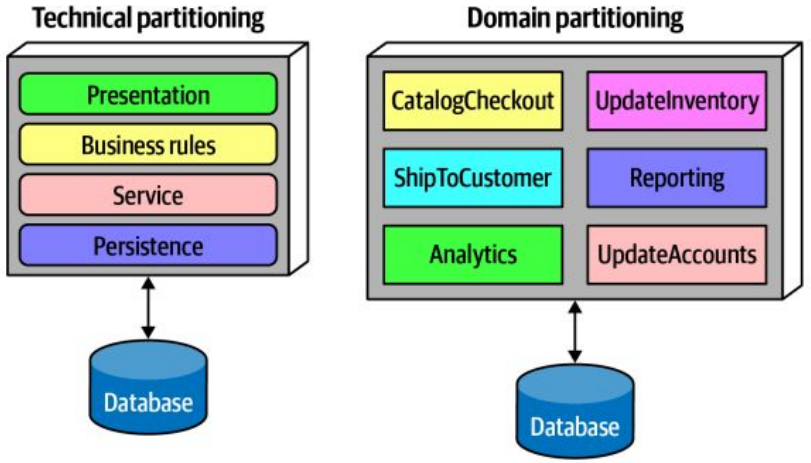
promotes the production, detection, consumption of, and reaction to events.

Space-Based Architecture

This resolves the issues of data

Architectural Partitioning

- Technical
  - Focuses on “**how**” the systems works
- Domain partitioning
  - Focuses on “**what**” the system does
  - Aligns better with business evolution and team ownership.



Agenda

1. Introduction and Fundamentals
2. **Architecture Styles**
3. Architecture Patterns
4. Software Design Patterns
5. Anti-Patterns and Metrics

Monolithic vs Distributed

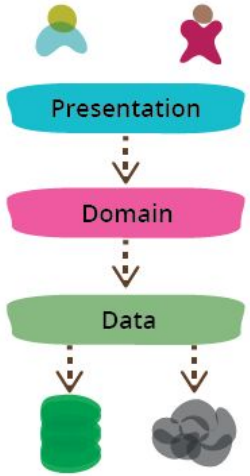
- Monolithic
  - Layered
  - Pipeline
  - Microkernel
- Distributed
  - Service-based
  - Event-driven
  - Microservices

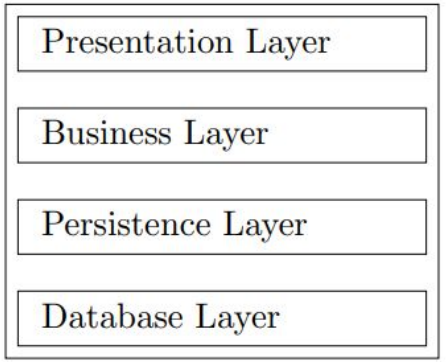
Agenda

1. Introduction and Fundamentals
2. Architecture Styles
    a. **Monolithic**
    b. Distributed
3. Architecture Patterns
4. Software Design Patterns
5. Anti-Patterns and Metrics

Layered

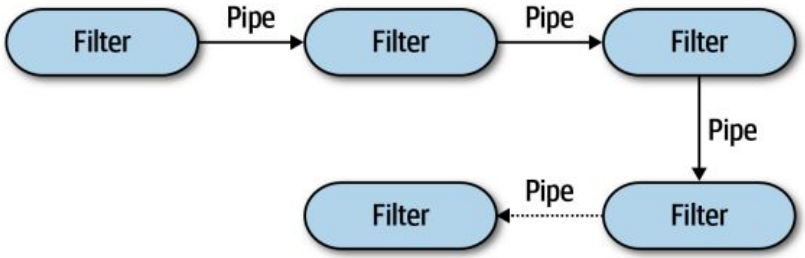
- Each layer **builds on the one below it**
- Clear **separation of responsibilities**
- Prevents **circular references**
- Unidirectional **dependencies**
- **Simplifies testing** and **evolution**





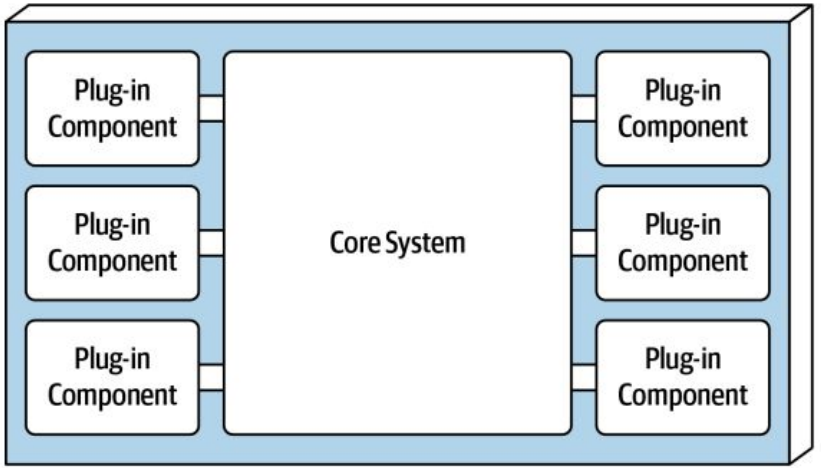
Pipeline

- Stream/batch processing with **deterministic steps** (ETL, media processing).
- Stages are **reusable**, **independently deployable** and **parallelizable**
- Trade-offs: **Back-pressure**, **need monitoring** and **global transactions** are hard.

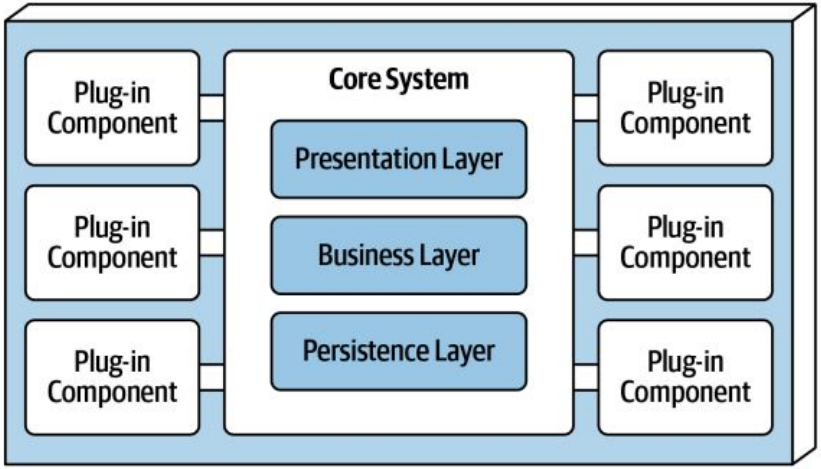


Microkernel

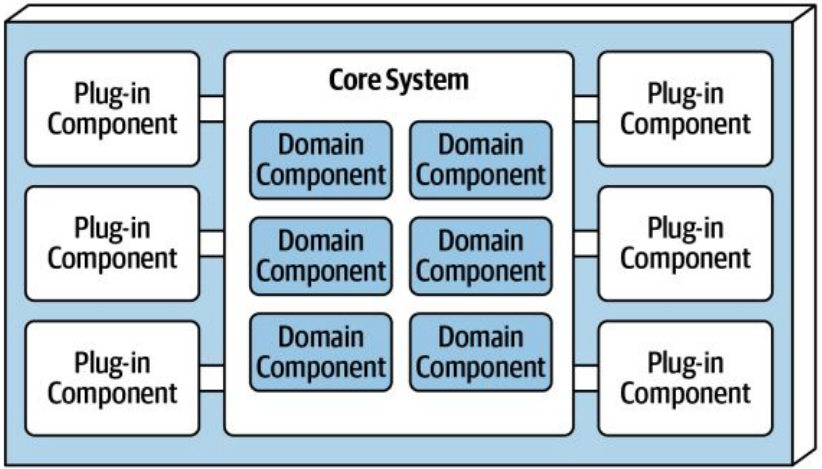
- Central core with **runtime extensibility** via independent components (like products or platforms)
- Trade-offs: **Contract design** is critical and **versioning/compatibility** overhead.



Microkernel



Layered Core System (Technically Partioned)



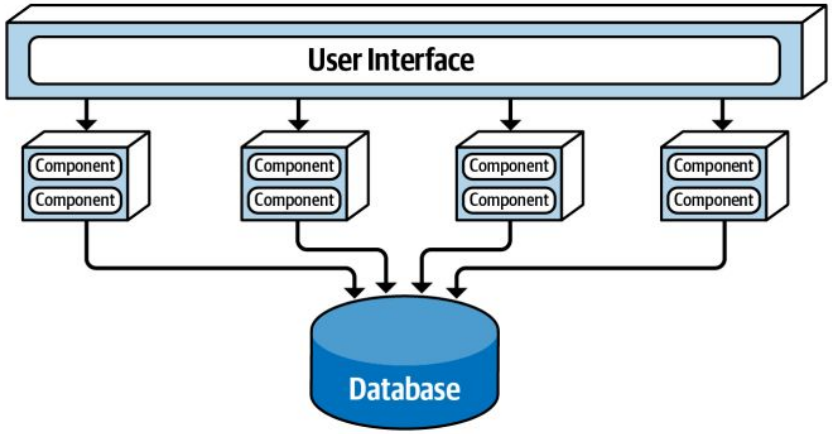
Modular Core System (Domain Partioned)

Agenda

1. Introduction and Fundamentals
2. Architecture Styles
    a. Monolithic
    b. Distributed
3. Architecture Patterns
4. Software Design Patterns
5. Anti-Patterns and Metrics

Service-based

- System built as a collection of **independent** and **reusable services**
- Uses **standard network protocols**
- Requires robust service design, **governance** and **security**
- Adds **complexity**



Event-Driven

- **Components communicate by producing and reacting to events** rather than direct calls
- Trade-offs: **Eventual consistency**, complex debugging/testing
- Best Used For: **Asynchronous** processing, **scalable**, **real-time** systems

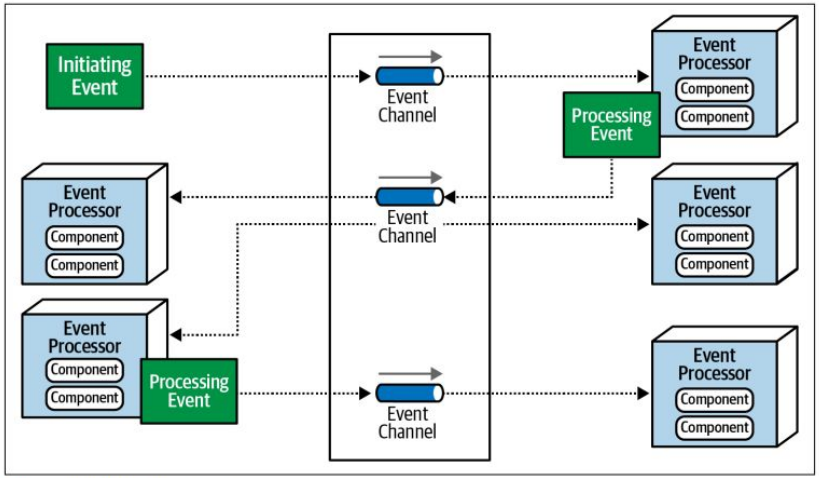


Figure 14-2. Broker topology

Event-Driven: Broker Topology

- Events flow through **lightweight message brokers**
- **High scalability** and **fault tolerance**
- Lacks **centralized workflow control**

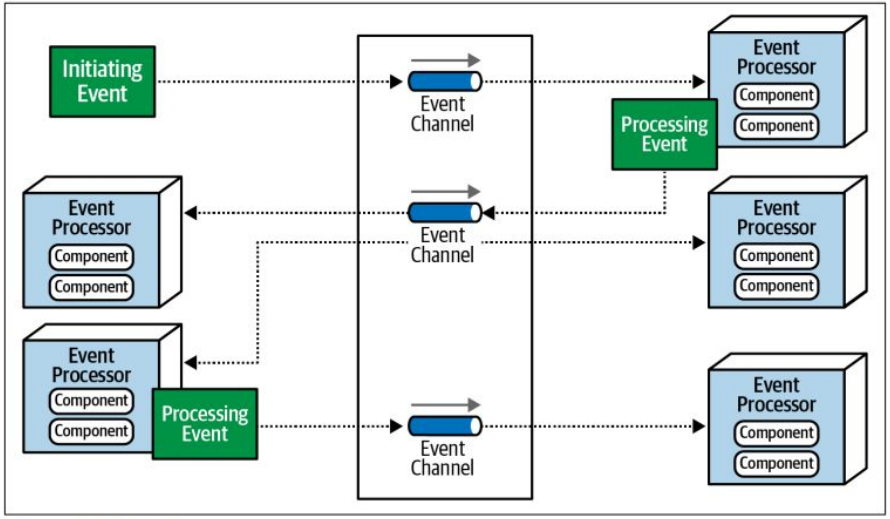


Figure 14-2. Broker topology

Event-Driven: Mediator Topology

- A **central mediator** coordinates event workflows
- **Better control** and **error handling**
- Adds **coupling** and **reduced performance**

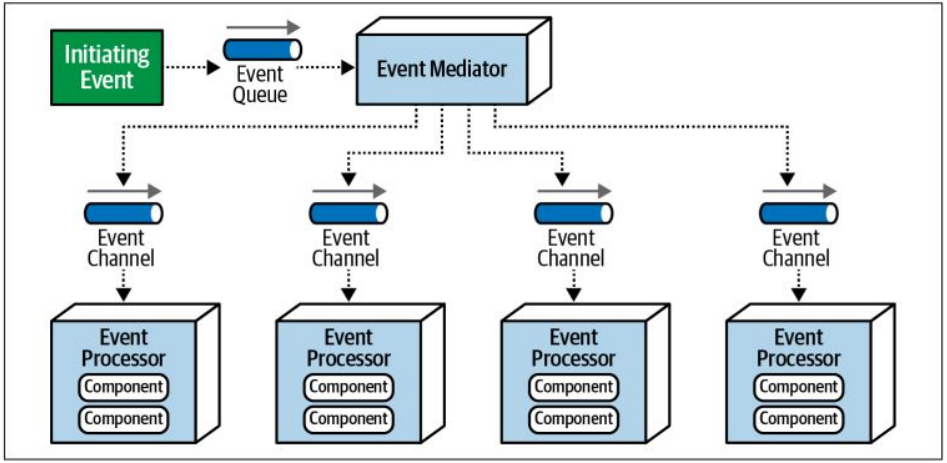
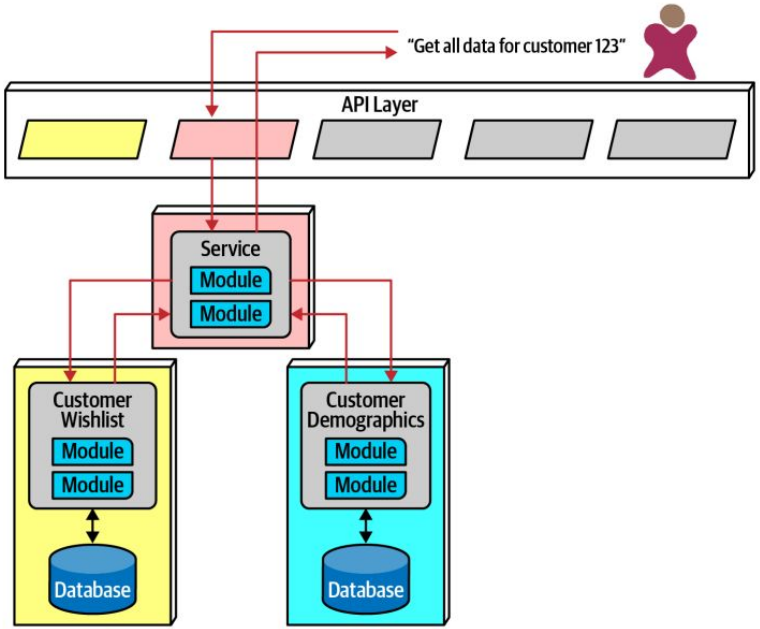


Figure 14-5. Mediator topology

Microservices

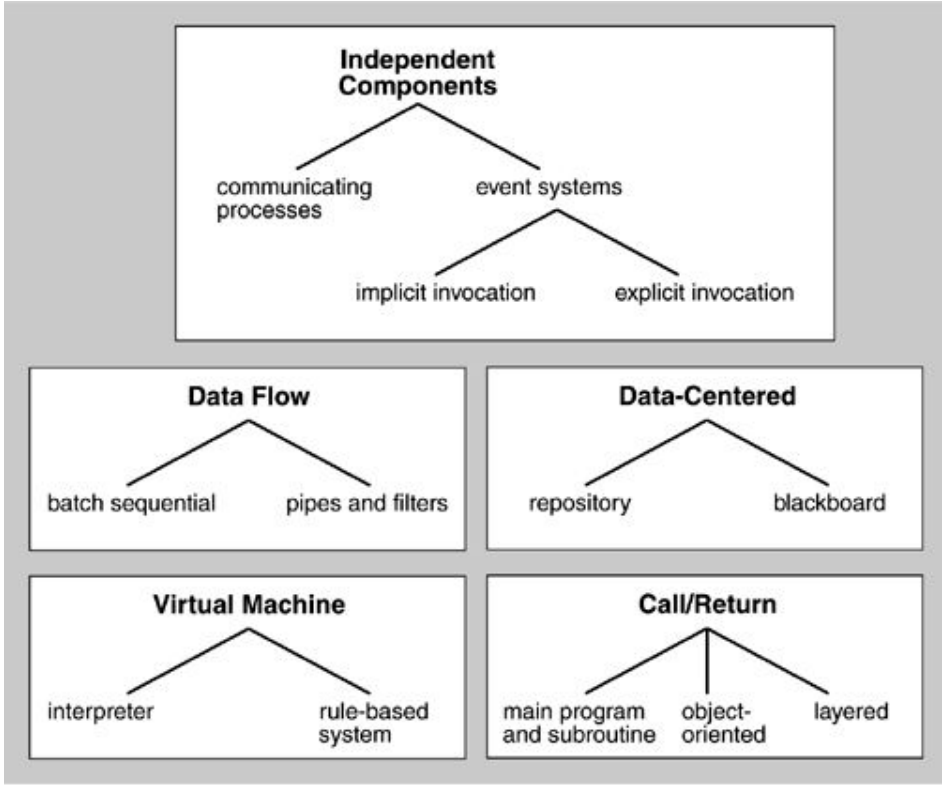
- Decomposes systems into **small, independent services** with a **bounded context** (business domain/workflow).
- Each service includes **its own data and dependencies**
- Supports **scalability**, **agility**, and **continuous delivery**
- Challenges**: coordination, degraded performance** due to network, **debugging** and difficult **data consistency**
- (too many variations to list here)



Agenda

1. Introduction and Fundamentals
2. Architecture Styles
3. **Architecture Patterns**
4. Software Design Patterns

Architecture Patterns



Agenda

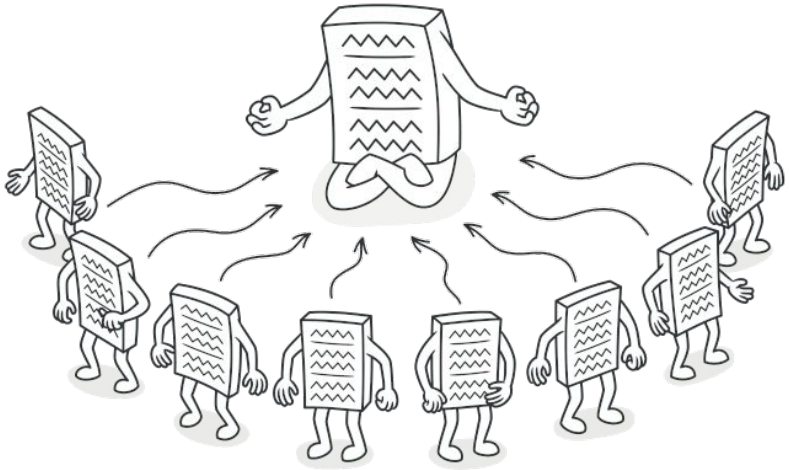
1. Introduction and Fundamentals
2. Architecture Styles
3. Architecture Patterns
4. **Software Design Patterns**

Categories

- Creational
  - Abstract Factory
  - Builder
  - Factory Method
  - Prototype
  - **Singleton**
- Structural
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - Facade
  - Flyweight
  - Proxy
- Behavioral
  - **Chain of Responsibility**
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - Observer
  - **State**
  - **Strategy**
  - **Template Method**
  - Visitor

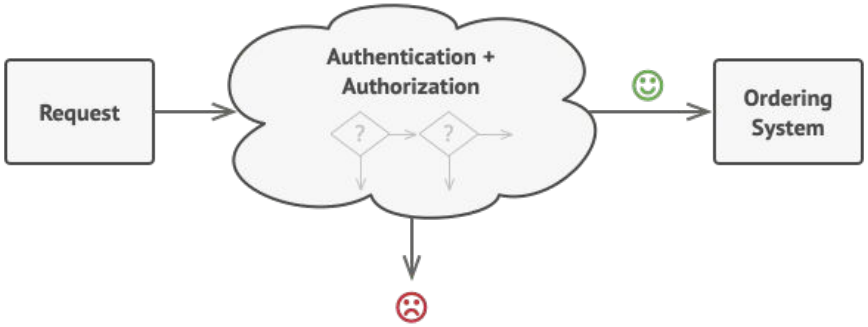
Singleton

- Problem: **Exactly one instance** is required (e.g., config, logger).
- Solution: Ensure a class has a **single global instance**.
- Trade-offs: +Easy access; −Hidden dependencies, testing difficulties.



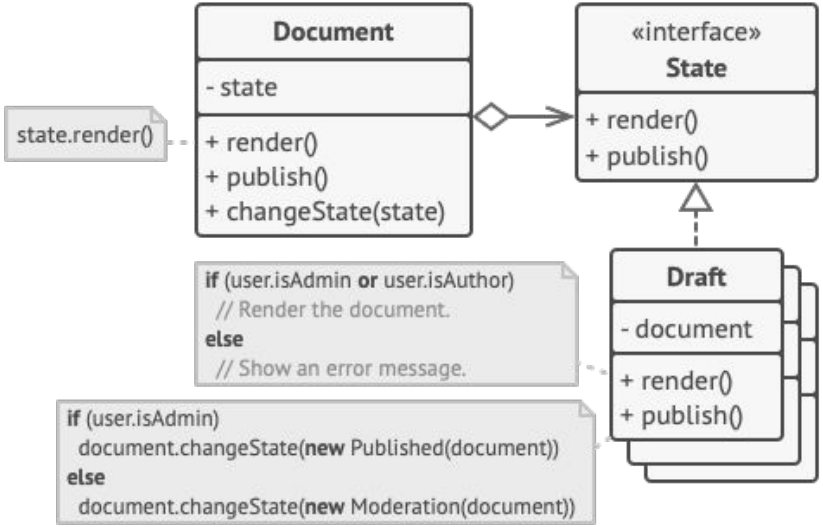
Chain of Responsibility

- Problem: **Hardwired handler logic creates tight coupling**.
- Solution: **Pass requests** along handlers until one handles it.
- Trade-offs: +Flexibility; −Uncertain handling/ordering.



State

- Problem: Many **conditional branches by state**.
- Solution: **Encapsulate state-specific** behavior and transitions.
- Trade-offs: +Cleaner logic; −More classes.

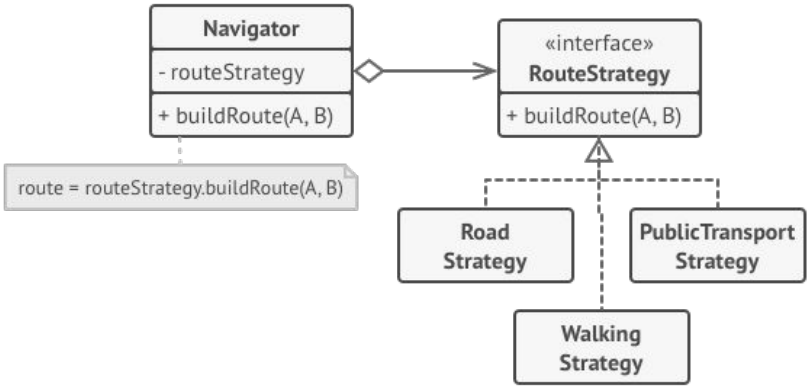


State


Draft
Published by user
Review failed
Publication expired
Moderation
Published by admin
Approved by admin
Published

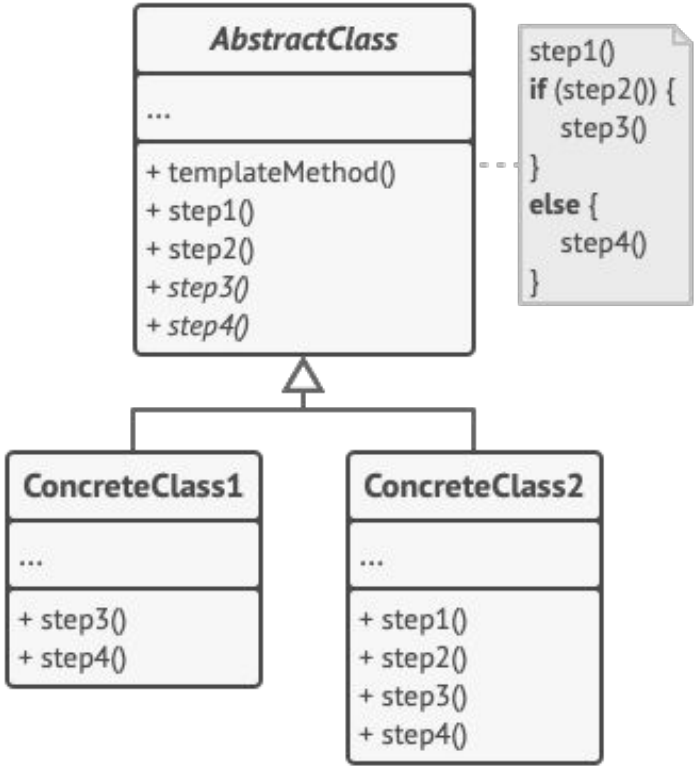
Strategy

- Problem: **Algorithm variants** scattered via conditionals.
- Solution: **Encapsulate each algorithm**; swap them at runtime behind a common interface.
- Trade-offs: +Open/closed; −More objects/config.
- Example: **Sorting strategies** (quick/merge/heap) behind an interface.

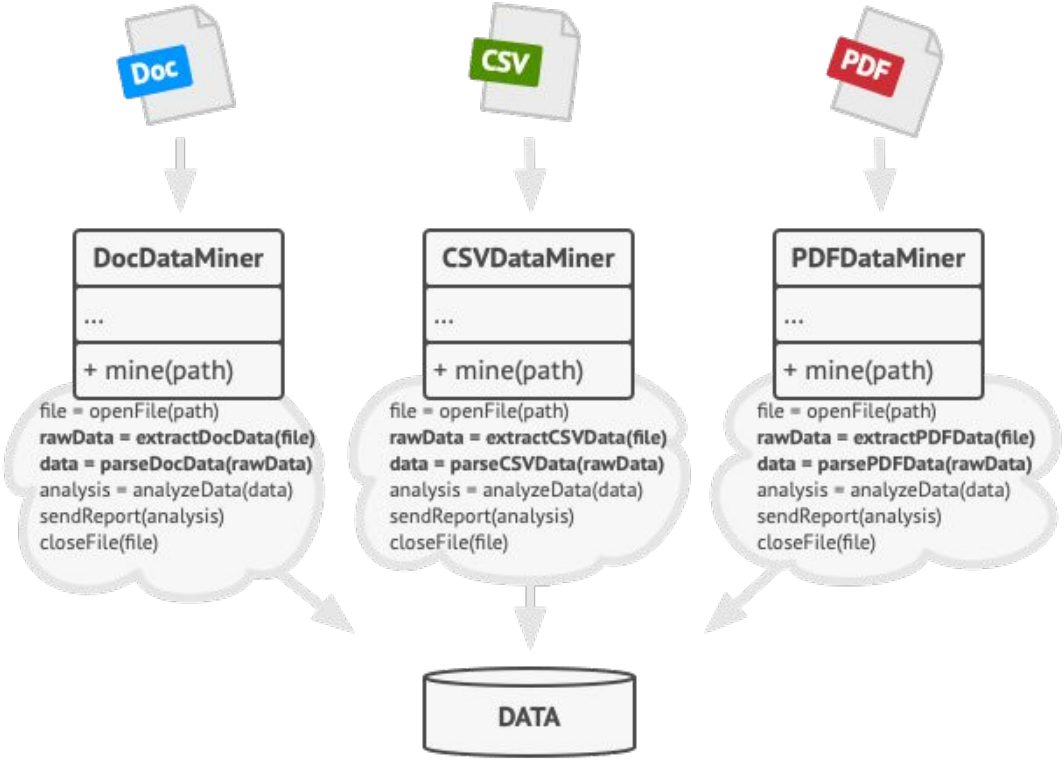


Template Method

- Problem: Algorithms **share shape but differ** in parts.
- Solution: Base class defines **template**; **subclasses the specifics**.
- Trade-offs: +Code reuse; −Inheritance coupling.



Template Method



Dudas?

References

- Design Patterns: Elements of Reusable Object-Oriented Software
- Fundamentals of Software Architecture
- https://martinfowler.com/eaaCatalog/
- https://arquisoft.github.io/
- https://csse6400.uqcloud.net/
- https://refactoring.guru/design-patterns
- https://www.kstar.com/indexproblem/18615.jhtml
- https://refactoring.guru/design-patterns/catalog

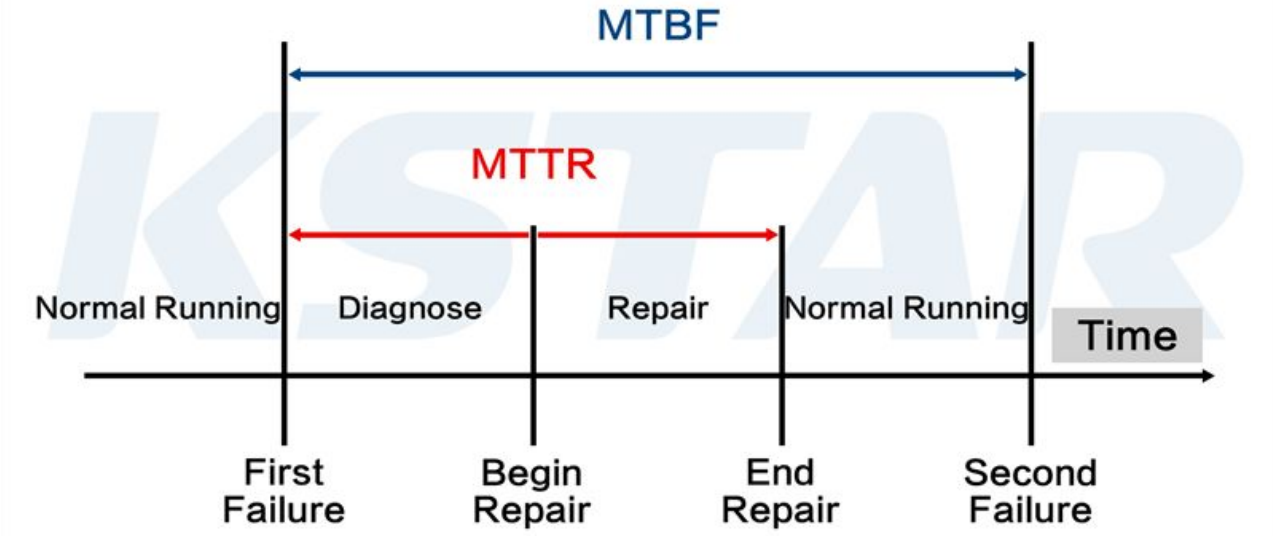
Extra Topics

- Representing systems with C4 model
- “Learning Domain-Driven Design”

Agenda

1. Introduction and Fundamentals
2. Architecture Styles
3. Architecture Patterns
4. Software Design Patterns
5. **Anti-Patterns and Metrics**

Recovery Metrics

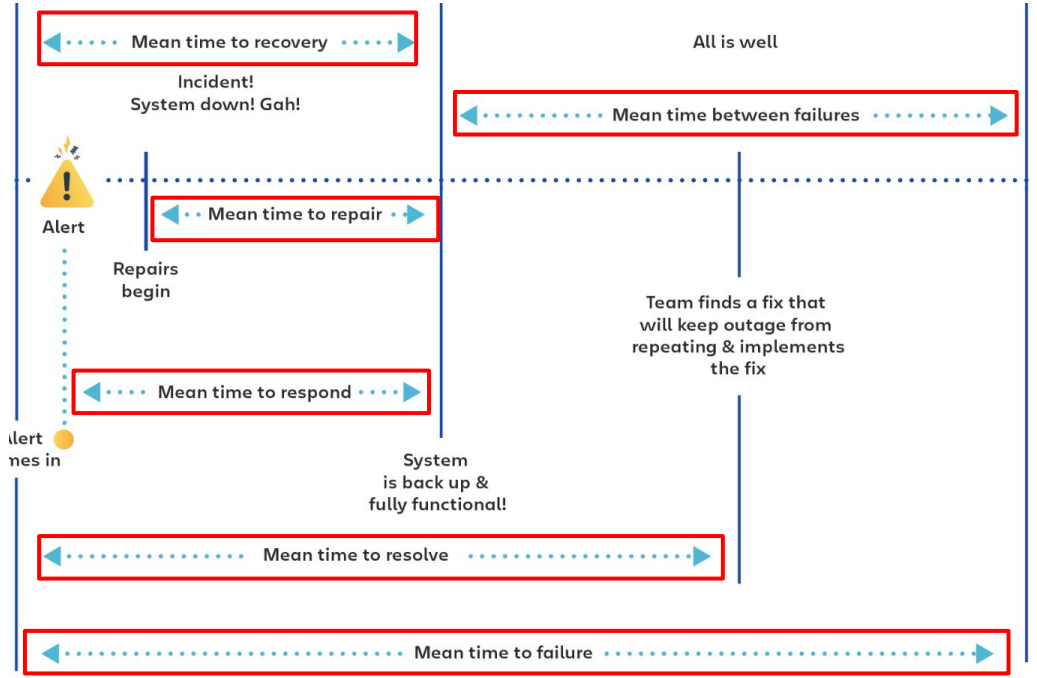


- MTTR: Mean time to repair
- MTBF: Mean time between repair

Recovery Metrics

- MTTF: To Recovery
- MTBF: Between Failures
- MTTR: to Repair / Response / Resolve
- MTBF: Between Failures

MT=Mean Time



DORA Metrics

	Elite	High	Medium	Low
Deployment frequency	**On-demand** (multiple deploys per day)	Between **once per day** and **once per week**	Between **once per week** and **once per month**	Between **once per month** and **once every six months**
Lead time for changes	Less than **one day**	Between **one day** and **one week**	Between **one week** and **one month**	Between **one month** and **six months**
Time to restore service	Less than **one hour**	Less than **one day**	Less than **one day**	Between **one week** and **one month**
Change failure rate	0-15%	0-15%	0-15%	46-60%

Domain-Driven Design

- Is a development approach that focuses on modeling software around the core business domain and its logic.

